Sous-équipe SE1 : Abdelouahab Alae – Hanahoum Hiba

RAPPORT DE SEMAINE 1 – INFRASTRUCTURE & MINI-CLUSTER

Objectifs :

Mettre en place la base technique du projet en :

- Définissant une architecture Big Data claire et documentée.
- Choisissant et justifiant les frameworks et les technologies principales.
- Préparant un environnement d'exécution cohérent.

☐ Le but est de fournir un socle stable pour que les autres sous-équipes puissent travailler dessus sans blocage.

1- Architecture

L'architecture globale du système combine deux modes de traitement complémentaires afin de gérer à la fois les données en temps réel et les données historiques. Les données viennent d'abord d'une source (une dataset), puis envoyées vers **Kafka**, qui agit comme point d'entrée et système de diffusion des flux. Une partie des données est stockée de manière durable dans **HDFS** pour permettre un traitement ultérieur et conserver l'historique complet. À partir de là, le système se divise en deux couches principales :

1-1- Couche Streaming (Temps réel) :

La couche streaming permet de traiter les données dès leur arrivée, Les données sont consommées directement depuis **Kafka** par des jobs **Apache Flink en mode streaming**, ce qui permet un traitement continu et instantané.

Une fois traitées, les données sont envoyées vers **ClickHouse**.

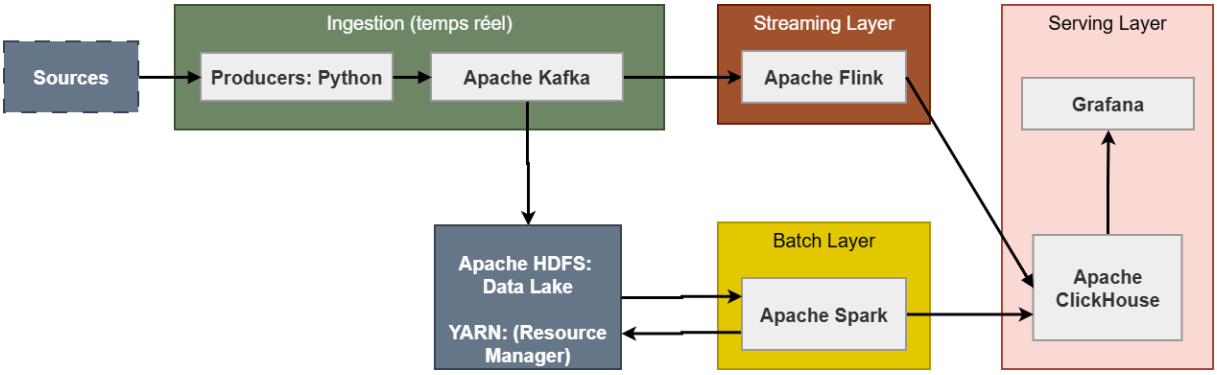
1-2- Couche Batch :

La couche batch permet de traiter les données de manière globale et différée à partir des données stockées dans HDFS.

Les jobs **Apache Spark en mode batch** lisent les données depuis HDFS et appliquent un processus **ETL progressif** afin de les rendre cohérentes et exploitables.

Une fois le traitement termine, les données peuvent être réécrites dans **HDFS** pour conserver l'historique et envoyées vers **ClickHouse** afin d'être exploitées dans la couche de visualisation et d'analyse.

Enfin, la visualisation permet d'exploiter les données traitées via des Dashboard et graphiques. Les données sont stockées dans Apache **ClickHouse**, puis interrogées par **Apache Graphana** pour créer des visualisations dynamiques



2- Choix des frameworks et technologies du système

Technologie/framework	Rôle dans le pipeline	Justification du choix
Hadoop 3 (HDFS + YARN)	**Infrastructure de base du système. assure le stockage distribué des données via HDFS et la gestion des ressources du cluster via YARN, permettant l'exécution des jobs de traitement.**	**Choisi pour sa capacité à stocker de grands volumes de données de manière fiable et distribuée, ainsi que pour sa bonne gestion des ressources. Il garantit la scalabilité, la tolérance aux pannes et une intégration facile avec les autres outils du système.**
Apache Kafka	**Bus de messagerie, assure l'ingestion et la diffusion des données en temps réel.**	**Choisi pour sa capacité à gérer de grands volumes de données en continu avec une faible latence. Il permet de découpler les producteurs et les consommateurs et garantit une transmission fiable et scalable des flux de données.**
Apache Spark	**Framework de traitement distribué.** **Utilisé pour exécuter des traitements batch sur de grandes données (nettoyage,**	**Choisi pour ses performances élevées dans le traitement de grands volumes de données stockées. Il permet d'exécuter des traitements distribués rapides et efficaces sur HDFS, avec une bonne optimisation des ressources. Son modèle basé sur le traitement par lots le rend particulièrement**

	transformation, agrégation)	adapté aux analyses historiques et aux opérations ETL complexes.
Apache Flink	Framework de traitement distribué. Assure le traitement des données en temps réel. Il consomme les flux depuis Kafka, applique des transformations et calcule des métriques en continu.	Choisi pour ses performances en traitement temps réel avec faible latence. Il permet un vrai streaming continu, une bonne gestion des événements et une grande scalabilité.
Apache ClickHouse	Base de données analytique OLAP orientée colonnes. Stocke les données traitées (streaming & batch) et les expose via une API REST et un port natif pour interrogation rapide.	Choisis pour ses performances OLAP exceptionnelles : son moteur de stockage en colonnes permet des agrégations 100 à 1000 fois plus rapides que les bases relationnelles classiques parfaitement compatible Docker (image officielle légère), 100% open source, et disposant d'une API REST complète facilitant l'interrogation avec Grafana. Particulièrement adapté aux requêtes analytiques sur de grands volumes de données en temps réel.
Graphana	Outil de visualisation / BI. (open source Edition) Il se connecte à Apache ClickHouse afin d'interroger les données traitées et de les présenter sous forme de tableaux de bord interactifs, avec rafraichissement automatique.	Choisi pour son écosystème de plugins riche (100+ visualisations : pie charts, gauges, geomaps, heatmaps), son rafraichissement temps reel configurable (5s a plusieurs heures), son alerting intégré basé sur des seuils, et sa compatibilité Docker native. 100% open source et sans licence payante, il permet de créer des dashboards interactifs et professionnels directement connectés à ClickHouse via un plugin officiel dédié.
Docker Compose	Orchestration du cluster local, permet de déployer et gérer plusieurs services en même temps (Kafka, Flink, Druid, Superset, etc..) a partir d'un seul fichier de configuration.	Choisis pour simplifier le déploiement de l'architecture en automatisant le lancement et la coordination des différents services.

3- Déploiement de l'infrastructure avec Docker Compose

Afin de déployer l'infrastructure Big Data de manière simple et reproductible, nous avons utilisé Docker Compose. Cet outil permet de définir et lancer plusieurs services conteneurisés à partir d'un seul fichier de configuration.

Dans notre cas, Docker Compose est utilisé pour mettre en place un mini-cluster Hadoop comprenant HDFS (NameNode et DataNodes) ainsi que YARN (ResourceManager, NodeManager et HistoryServer).
Chaque composant du cluster est exécuté dans un conteneur indépendant, ce qui permet d'isoler les services tout en assurant leur communication via un réseau commun.
Le cluster est composé de :

- Un **NameNode**, responsable de la gestion des métadonnées HDFS.
- Deux **DataNodes**, permettant de simuler un stockage distribué et assurer la réplication des données.
- Un **ResourceManager** et un **NodeManager** pour la gestion et l'exécution des tâches.
- Un **HistoryServer** pour le suivi des jobs.

L'utilisation de volumes Docker permet de garantir la persistance des données même après l'arrêt des conteneurs. De plus, un fichier de configuration externe (**hadoop.env**) est utilisé afin de centraliser les paramètres Hadoop, facilitant ainsi la maintenance et la modification de la configuration.
Enfin, cette approche permet de reproduire facilement l'environnement sur une autre machine, ce qui est essentiel pour le travail collaboratif entre les différentes sous-équipes.

Élément	Rôle
NameNode	**Gestion des métadonnées HDFS**
DataNodes	**Stockage distribué des données**
ResourceManager	**Gestion des ressources YARN**
NodeManager	**Exécution des tâches**
HistoryServer	**Historique des jobs**

Volumes	**Persistance des données**
Réseau Docker	**Communication entre services**